Lab Report – EE-170L Computer Programming (Lab 04)

NAME	NAEEM UL HAQ
ID	0665
SEMESTER	2nd
LAB	4 (programming)
DATE	5/3/2026

Objective

The purpose of this lab is to:

- Understand and use the switch statement in C++.
- Apply the break statement to control program flow.
- Use the continue statement inside loops.
- Differentiate between break and continue.
- Develop simple menu-driven programs.

Theory

Switch Statement

- The switch statement is used to select one block of code from multiple options based on the value of a variable.
- It is an alternative to multiple if-else statements when checking a single variable.
- Each case in a switch corresponds to a constant integral expression.
- The default case handles unmatched values.

Syntax:

```
switch (expression) {
   case value1:
      // code
```

```
        break;
    case value2:
        // code
        break;
    default:
        // code
}
```

Break Statement

- Ends the execution of the current case.
- Prevents fall-through into subsequent cases.
- Without, execution continues until the next break or the end of the switch block.

Continue Statement

- Used inside loops.
- Skips the current iteration and moves to the next iteration of the loop.

Programs and Implementation

Program 1: Day of the Week

```
#include <iostream>
using namespace std;
```

```
int main() {
    int day;
    cout << "Enter day number (1-7): ";
    cin >> day;

    switch(day) {
        case 1: cout << "Monday"; break;
        case 2: cout << "Tuesday"; break;
        case 3: cout << "Wednesday"; break;
        case 4: cout << "Thursday"; break;
        case 5: cout << "Friday"; break;
        case 6: cout << "Saturday"; break;
        case 7: cout << "Sunday"; break;
        default: cout << "Invalid day!";
    }
    return 0;
}
```

Sample output:

Enter day number (1-7): 3 Wednesday

Program 2: Grade Evaluation

```
#include <iostream>
using namespace std;

int main () {
    char grade;
    cout << "Enter grade (A, B, C, D, F): ";
    cin >> grade;

    switch(grade) {
        case 'A': cout << "Excellent"; break;
        case 'B': cout << "Very Good"; break;
        case 'C': cout << "Good"; break;
        case 'D': cout << "Pass"; break;
        case 'F': cout << "Fail"; break;
        default: cout << "Invalid Grade!";
    }
    return 0;
}
```

Sample output:

Enter grade (A, B, C, D, F): B very good

Program 3: Conversion Menu

```
#include <iostream>
using namespace std;

int main() {
    int choice;
    double value;

    cout << "Menu:\n";
    cout << "1 -> Kilometers to Meters\n";
    cout << "2 -> Meters to Centimeters\n";
    cout << "3 -> Kilograms to Grams\n";
    cout << "4 -> Celsius to Fahrenheit\n";
    cout << "Enter your choice: ";
    cin >> choice;

    switch(choice) {
        case 1:
            cout << "Enter kilometers: ";
            cin >> value;
            cout << "Meters = " << value * 1000;
            break;
        case 2:
```

```
        cout << "Enter meters: ";
        cin >> value;
        cout << "Centimeters = " << value * 100;
        break;
    case 3:
        cout << "Enter kilograms: ";
        cin >> value;
        cout << "Grams = " << value * 1000;
        break;
    case 4:
        cout << "Enter Celsius: ";
        cin >> value;
        cout << "Fahrenheit = " << (value * 9/5) + 32;
        break;
    default:
        cout << "Invalid choice!";
    }
    return 0;
}
```

Output sample:

Menu:

1 -> Kilometers to Meters

2 -> Meters to Centimeters

3 -> Kilograms to Grams

4 -> Celsius to Fahrenheit

Enter your choice: 4

Enter Celsius: 25

Fahrenheit = 77

Results

- The switch statement successfully selects among multiple options.
- The break statement prevents fall-through and ensures correct program flow.
- The default case handles invalid inputs.
- Menu-driven programs can be efficiently implemented using switch.

Conclusion

In this lab, we learned:

- How to implement switch-case statements in C++.
- The importance of break in controlling execution.
- The role of default in handling unmatched cases.

- Practical applications of switch in building calculators, grade evaluators, and conversion menus.